Praktikum 4
Register Instruksi dan Accumulator

NRP	: 1. 3223600038
	2. 3223600041
	3. 3223600048
	4. 3223600050
	5. 3223600058
Nama	: 1. Karolus Wira Wicaksana.....
	2. Akmal Rafiuddin.....
	3. Moch. Faris Afif.....
	4. Labib Al Afaf..
	5. Ihsanta Zaki Sanjaya.....
Kelas	: 2 D4 Teknik Komputer B
Matakuliah	: Praktikum Desain dadn Organisasi Komputer
Tanggal	: Rabu, 26 Maret 2025
Dosen	: Dr. Setiawardhana, ST. MT

D4 TEKNIK KOMPUTER

POLITEKNIK ELEKTRONIKA NEGERI SURABAYA

2025

Register Instruksi dan Accumulator

4.1. Tujuan:

a. Mahasiswa dapat menjelaskan dan membuat program Register Instruksi.

b. Mahasiswa dapat menjelaskan dan membuat program Accumulator.

4.2. Dasar Teori:

A. Register Instruksi

Register instruksi atau Instruction Register yang disingkat **IR** merupakan bagian dari unit kendali. Untuk mengambil sebuah instruksi dari memori, komputer melakukan operasi membaca memori. Dalam operasi ini isi dari lokasi memori yang ditunjuk alamatnya ditempatkan pada bus W. Pada waktu yang sama, register instruksi disiapkan untuk pengisian pada tepi positif dari sinyal detak (clock) berikutnya.

Isi register instruksi dibagi menjadi dua nibble. Nibble bagian atas merupakan keluaran dua keadaan yang langsung dikirimkan kepada blok “Pengendali Pengurut”. Nibble bagian bawah adalah keluaran tiga keadaan yang dapat dibaca (ditempatkan) pada bus W bilamana diperlukan.

B. Accumulator

Akumulator disingkat dengan **A** adalah sebuah register buffer yang menyimpan jawaban sementara selama komputer beroperasi. Dalam Gambar 1 diperlihatkan bahwa akumulator mempunyai dua macam keluaran. Keluaran dua keadaan secara langsung diteruskan ke bagian Arithmetic Logic Unit. Keluaran tiga-keadaan dikirimkan kepada bus W. Karena itu kata 8-bit dari akumulator secara terus menerus menggerakkan rangkaian Arithmetic Logic Unit; dan kata yang sama juga muncul pada bus W bilamana E_A tinggi.

4.3. Tugas Pendahuluan:

a. Tuliskan tujuan praktikum

Tujuan dalam praktikum ini adalah agar mahasiswa dapat memahami dan mempraktikkan cara kerja dari dua komponen penting dalam sistem komputer, yaitu Register Instruksi (Instruction Register) dan Accumulator. Dengan praktikum ini, mahasiswa diharapkan mampu menjelaskan fungsi masing-masing komponen serta mengimplementasikan program yang berkaitan menggunakan VHDL.

b. Apa yang anda ketahui tentang Register Instruksi

Register Instruksi (Instruction Register atau IR) adalah bagian dari unit kendali dalam sistem komputer yang berfungsi untuk menyimpan instruksi yang sedang dieksekusi oleh prosesor. Saat sebuah instruksi diambil dari memori, data instruksi tersebut

dimasukkan ke dalam register ini. IR akan memecah isi instruksi menjadi dua bagian: bagian atas digunakan untuk mengatur langkah-langkah eksekusi selanjutnya melalui blok pengendali pengurut, sedangkan bagian bawah dapat digunakan kembali bila diperlukan oleh sistem.

c. Apa yang anda ketahui tentang Pengendali Pengurut

Pengendali Pengurut merupakan bagian dari sistem kontrol dalam komputer yang digunakan untuk mengatur urutan langkah-langkah dalam eksekusi instruksi. Komponen ini menerima sinyal dari register instruksi dan kemudian menghasilkan sinyal kontrol yang diperlukan untuk mengarahkan jalannya operasi di dalam komputer, seperti memindahkan data atau melakukan perhitungan. Bisa dibilang, pengendali pengurut adalah otaknya yang mengatur kapan dan bagaimana bagian-bagian lain dari komputer bekerja.

4.4. Percobaan:

a. Membuat program Register Instruksi

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity IR is
    Port ( CLK : in  STD_LOGIC;
           CLR : in  STD_LOGIC;
           nLi : in  STD_LOGIC;
           nEi : in  STD_LOGIC;
           inIR : in  STD_LOGIC_VECTOR (7 downto 0);
           outIRlow : out  STD_LOGIC_VECTOR (3 downto 0);
           outIRhigh : out  STD_LOGIC_VECTOR (3 downto
           0));
end IR;

architecture Behavioral of IR is
      signal regBuff : STD_LOGIC_VECTOR (7 downto 0);
begin
      process (CLK,CLR)
      begin
            if rising_edge(CLK) then
                  if CLR = '1' then
                        regBuff <="00000000";
                        outIRlow <="ZZZZ";
                  elsif nLi='0' then
                        regBuff <= inIR;
                        outIRlow <= "ZZZZ";
                  end if;
            end if;
      end process;
      outIRlow <= regBuff(3 downto 0) when nEi='0' else "ZZZZ";
      outIRhigh <= regBuff(7 downto 4);
end Behavioral;
```

b. Membuat program Accumulator

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity regA is
    Port ( CLK     : in  STD_LOGIC;
           nLa     : in
           STD_LOGIC; Ea      :
           in  STD_LOGIC;
           inA     : in   STD_LOGIC_VECTOR (7 downto
           0); outA       : out  STD_LOGIC_VECTOR (7
           downto 0);
          regACC : out  STD_LOGIC_VECTOR (7 downto 0));
end regA;

architecture Behavioral of regA is
      signal sigA : STD_LOGIC_VECTOR (7 downto 0);
begin
      process (CLK)
      begin
            if rising_edge(CLK)
                  then if nLa='0'
                  then
                        sigA    <=
                  inA; end if;
            end if;
      end process;
      outA <= sigA when Ea='1' else "ZZZZZZZZ";
      regACC <= sigA;
end Behavioral;
```

4.5. Latihan:

a. Buatlah simulasi program Register Instruksi.

b. Buatlah simulasi program Accumulator.

4.6. Percobaan

a. Register Intruksi

- Program:

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity IR is
    Port ( CLK : in  STD_LOGIC;
           CLR : in  STD_LOGIC;
           nLi : in  STD_LOGIC;
           nEi : in  STD_LOGIC;
           inIR : in  STD_LOGIC_VECTOR (7 downto 0);
           outIRlow : out  STD_LOGIC_VECTOR (3 downto 0);
           outIRhigh : out  STD_LOGIC_VECTOR (3 downto 0));
end IR;

architecture Behavioral of IR is
 signal regBuff : STD_LOGIC_VECTOR (7 downto 0);
begin
 process (CLK,CLR)
 begin
  if rising_edge(CLK) then
   if CLR = '1' then
    regBuff <="00000000";
    outIRlow <="ZZZZ";
   elsif nLi='0' then

   if CLR = '1' then
    regBuff <="00000000";
    outIRlow <="ZZZZ";
   elsif nLi='0' then
    regBuff <= inIR;
    outIRlow <= "ZZZZ";
   end if;
  end if;
 end process;
 outIRlow <= regBuff(3 downto 0) when nEi='0' else "ZZZZ";
 outIRhigh <= regBuff(7 downto 4);
end Behavioral;
```

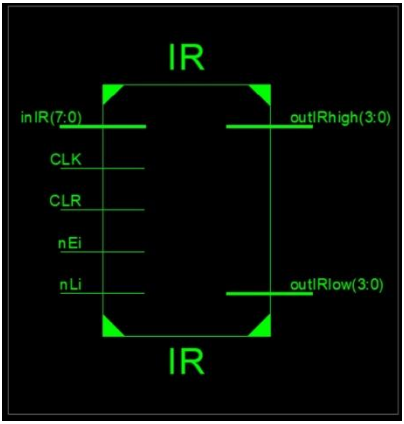
- Testbench:

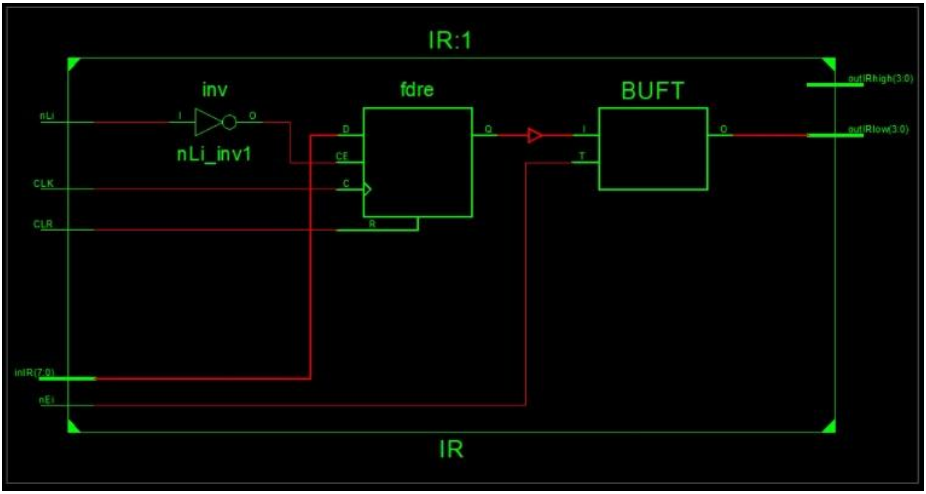
```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity IR_tb is
-- Testbench tidak memiliki port
end IR_tb;

architecture Behavioral of IR_tb is

    -- Komponen yang akan diuji
    component IR
        Port (
            CLK : in  STD_LOGIC;
            CLR : in  STD_LOGIC;
            nLi : in  STD_LOGIC;
            nEi : in  STD_LOGIC;
            inIR : in  STD_LOGIC_VECTOR (7 downto 0);
            outIRlow : out  STD_LOGIC_VECTOR (3 downto
0);
            outIRhigh : out  STD_LOGIC_VECTOR (3 downto
0)
        );
    end component;

    -- Sinyal internal untuk menghubungkan ke UUT (Unit
Under Test)
    signal CLK_tb     : STD_LOGIC := '0';
    signal CLR_tb     : STD_LOGIC := '0';
    signal nLi_tb     : STD_LOGIC := '1';
    signal nEi_tb     : STD_LOGIC := '1';
    signal inIR_tb    : STD_LOGIC_VECTOR (7 downto 0) :=
(others => '0');
    signal outIRlow_tb  : STD_LOGIC_VECTOR (3 downto 0);
    signal outIRhigh_tb : STD_LOGIC_VECTOR (3 downto 0);

    -- Clock period
    constant CLK_PERIOD : time := 10 ns;

begin
```

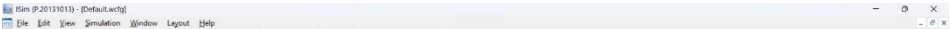
```
    -- Instance dari unit yang diuji
    uut: IR
        Port map (
            CLK => CLK_tb,
            CLR => CLR_tb,
            nLi => nLi_tb,
            nEi => nEi_tb,
            inIR => inIR_tb,
            outIRlow => outIRlow_tb,
            outIRhigh => outIRhigh_tb
        );

    -- Clock process
    clk_process : process
    begin
        while now < 200 ns loop
            CLK_tb <= '0';
            wait for CLK_PERIOD / 2;
            CLK_tb <= '1';
            wait for CLK_PERIOD / 2;
        end loop;
        wait;
    end process;

    -- Stimulus process
    stim_proc: process
    begin
        -- Initial delay
        wait for 20 ns;

        -- Reset aktif
        CLR_tb <= '1';
        wait for CLK_PERIOD;
        CLR_tb <= '0';

        -- Load nilai pertama
        inIR_tb <= "10101010";
        nLi_tb <= '0';  -- aktif low
        wait for CLK_PERIOD;
        nLi_tb <= '1';

        -- Enable output
        nEi_tb <= '0';  -- aktif low
        wait for CLK_PERIOD;

        -- Load nilai kedua
```

```
        inIR_tb <= "11001100";
        nLi_tb <= '0';
        nEi_tb <= '1';  -- disable output dulu
        wait for CLK_PERIOD;
        nLi_tb <= '1';

        -- Enable kembali
        nEi_tb <= '0';
        wait for CLK_PERIOD;

        -- Selesai
        wait;
    end process;

end Behavioral;
```

- Hasil:







b. Accumulator

- Program:

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity regA is
    Port ( CLK  : in  STD_LOGIC;
           nLa  : in  STD_LOGIC;
           Ea : in  STD_LOGIC;
           inA  : in   STD_LOGIC_VECTOR (7 downto 0);
           outA  : out  STD_LOGIC_VECTOR (7 downto 0);
                   regACC : out  STD_LOGIC_VECTOR (7 downto
0));
end regA;

architecture Behavioral of regA is
     signal sigA : STD_LOGIC_VECTOR (7 downto 0);

begin
     process (CLK)
     begin
           if rising_edge(CLK) then
                 if nLa='0' then
                       sigA <= inA;
                 end if;
           end if;
     end process;
     outA <= sigA when Ea='1' else "ZZZZZZZZ";
     regACC <= sigA;
end Behavioral;
```

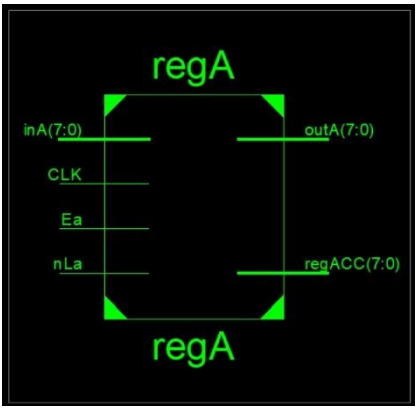
- Testbench:

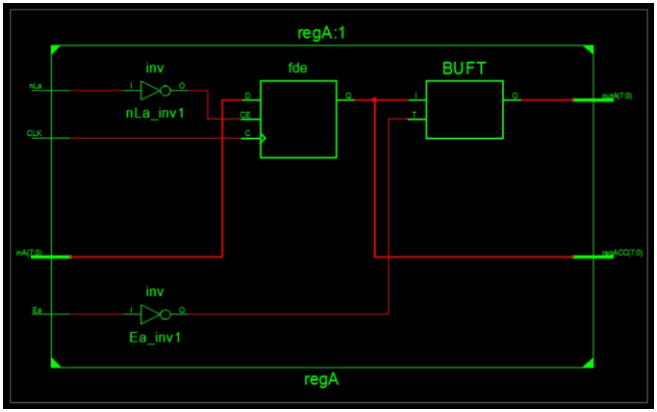
```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity regA_tb is
-- Tidak ada port di testbench
end regA_tb;

architecture Behavioral of regA_tb is

    -- Komponen yang diuji (UUT)
    component regA
```

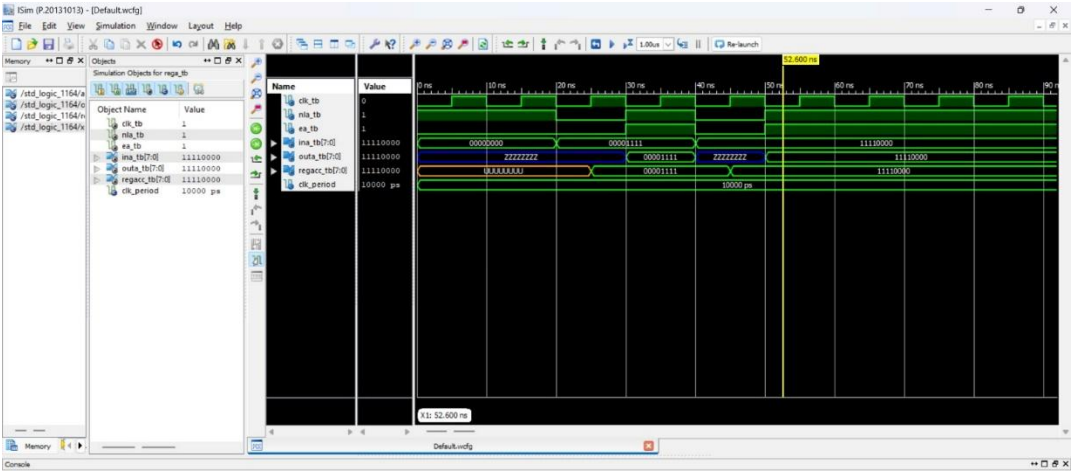
```
Port (
            CLK   : in  STD_LOGIC;
            nLa   : in  STD_LOGIC;
            Ea    : in  STD_LOGIC;
            inA   : in  STD_LOGIC_VECTOR (7 downto 0);
            outA  : out STD_LOGIC_VECTOR (7 downto 0);
            regACC : out STD_LOGIC_VECTOR (7 downto 0)
        );
    end component;

    -- Sinyal untuk menghubungkan testbench ke UUT
    signal CLK_tb    : STD_LOGIC := '0';
    signal nLa_tb    : STD_LOGIC := '1';
    signal Ea_tb     : STD_LOGIC := '0';
    signal inA_tb    : STD_LOGIC_VECTOR(7 downto 0) :=
(others => '0');
    signal outA_tb   : STD_LOGIC_VECTOR(7 downto 0);
    signal regACC_tb : STD_LOGIC_VECTOR(7 downto 0);

    constant CLK_PERIOD : time := 10 ns;

begin

    -- Instansiasi unit yang diuji
    uut: regA
        Port map (
            CLK => CLK_tb,
            nLa => nLa_tb,
            Ea => Ea_tb,
            inA => inA_tb,
            outA => outA_tb,
            regACC => regACC_tb
        );

    -- Clock process
    clk_process : process
    begin
        while now < 200 ns loop
            CLK_tb <= '0';
            wait for CLK_PERIOD / 2;
            CLK_tb <= '1';
            wait for CLK_PERIOD / 2;
        end loop;
        wait;
    end process;

    -- Stimulus process
    stim_proc: process
    begin
        -- Delay awal
```

```
        wait for 20 ns;

        -- Load data pertama ke akumulator
        inA_tb <= "00001111";
        nLa_tb <= '0';  -- aktif low
        wait for CLK_PERIOD;
        nLa_tb <= '1';

        -- Enable output
        Ea_tb <= '1';
        wait for CLK_PERIOD;

        -- Load data kedua
        inA_tb <= "11110000";
        nLa_tb <= '0';
        Ea_tb <= '0'; -- matikan output sementara
        wait for CLK_PERIOD;
        nLa_tb <= '1';

        -- Enable kembali
        Ea_tb <= '1';
        wait for CLK_PERIOD;

        -- Selesai
        wait;
    end process;

end Behavioral;
```

- Hasil:


regA
inA(7:0)
outA(7:0)
CLK
Ea
nLa
regACC(7:0)
regA


regA:1
inv
fde
BUFT
nLa
nLa_inv1
CLK
inA(7:0)
Ea
inv
Ea_inv1
regA


iSim (P.20131013) - [Default.wcfg]
File Edit View Simulation Window Layout Help
Simulation Objects for rega_tb
Object Name
Value
clk_tb
nla_tb
ea_tb
ina_tb[7:0]
outa_tb[7:0]
regacc_tb[7:0]
clk_period
10000 ps
Name
Value
52.600 ns
X1: 52.600 ns
Console

4.7. Analisa

a. Program Register Instruksi

Program VHDL ini mengimplementasikan sebuah modul register yang disebut "IR" dengan input dan output sebagai berikut:

Input:

- CLK: sinyal clock.
- CLR: sinyal reset.
- nLi: sinyal kontrol yang digunakan untuk mengaktifkan loading data ke dalam register.
- nEi: sinyal kontrol yang digunakan untuk mengaktifkan keluaran data dari register.
- inIR: sinyal data input 8-bit.

Output:

- outIRlow: sinyal data output 4-bit yang merepresentasikan 4-bit terendah dari register.
- outIRhigh: sinyal data output 4-bit yang merepresentasikan 4-bit tertinggi dari register.

Perilaku modul didefinisikan dalam arsitektur "Behavioral". Di dalam arsitektur, terdapat sebuah proses yang sensitif terhadap sinyal clock dan reset. Ketika sinyal clock naik, proses tersebut memeriksa nilai sinyal reset. Jika nilai sinyal reset tinggi (yaitu 1), maka register

direset dengan mengatur regBuff menjadi "00000000" dan outIRlow menjadi "ZZZZ".

Jika nilai sinyal reset rendah (yaitu 0), proses tersebut memeriksa nilai sinyal nLi. Jika nilai sinyal nLi rendah (yaitu 0), data input (inIR) dimuat ke dalam register (regBuff), dan outIRlow diatur menjadi "ZZZZ".

Pada akhir proses, 4 bit terendah dari register (regBuff(3 downto 0)) ditugaskan ke outIRlow ketika nEi rendah (yaitu 0) untuk mengaktifkan keluaran. Jika tidak, outIRlow diatur menjadi "ZZZZ". Demikian pula, 4 bit tertinggi dari register (regBuff(7 downto 4))
ditugaskan ke outIRhigh untuk memberikan keluaran.

b. Program Accumulator

Kode VHDL ini mengimplementasikan sebuah modul register yang disebut "regA" dengan input dan output sebagai berikut:

Input:

- CLK: sinyal clock.
- nLa: sinyal kontrol yang digunakan untuk mengaktifkan loading data ke dalam register.
- Ea: sinyal kontrol yang digunakan untuk mengaktifkan keluaran data dari register.
- inA: sinyal data input 8-bit.

Output:

- outA: sinyal data output 8-bit yang merepresentasikan nilai terbaru dari register ketika Ea=1.
- regACC: sinyal data output 8-bit yang merepresentasikan nilai terbaru dari register.

Perilaku modul didefinisikan dalam arsitektur "Behavioral". Di dalam arsitektur, terdapat sebuah proses yang sensitif terhadap sinyal clock. Ketika sinyal clock naik, proses tersebut memeriksa nilai sinyal nLa. Jika nilai sinyal nLa rendah (yaitu 0), data input (inA) dimuat ke dalam register (sigA). Pada akhir proses, keluaran dari register ditugaskan ke outA dan regACC. Jika sinyal kontrol Ea tinggi (yaitu 1), maka outA akan merepresentasikan nilai terbaru dari register (sigA). Jika tidak, outA diatur menjadi "ZZZZZZZZ".

Dalam konteks akumulator, nilai dalam register (sigA) dapat dianggap sebagai nilai yang sedang diakumulasi. Nilai ini selalu tersedia di dalam register dan dapat diakses kapan saja melalui sinyal output regACC. Sinyal output outA digunakan untuk memberikan hasil akumulasi terbaru ketika diperlukan.

4.8. Kesimpulan

Percobaan pertama mengimplementasikan sebuah modul register yang disebut "IR" dengan dua output port 4-bit (outIRlow dan outIRhigh) dan input port 8-bit (inIR). Modul register akan memuat data dari input port saat sinyal kontrol (nLi) rendah, dan output port akan merefleksikan nilai terbaru dari register saat sinyal kontrol lain (nEi) juga rendah. Register akan direset menjadi 0 saat sinyal clear (CLR) tinggi.

Percobaan kedua mengimplementasikan sebuah modul akumulator yang disebut "regA" dengan input port 8-bit (inA) dan dua output port 8-bit (outA dan regACC). Modul akumulator akan memuat data dari input port saat sinyal kontrol (nLa) rendah dan mengeluarkan nilai akumulator saat sinyal output (regACC) dipanggil. Output port outA akan merefleksikan nilai terbaru dari akumulator saat sinyal kontrol lain (Ea) tinggi, dan output akan diatur menjadi "ZZZZZZZZ" saat Ea rendah.

Secara ringkas, kedua percobaan mengimplementasikan modul register dan akumulator dengan fungsionalitas yang berbeda, tetapi keduanya menggunakan sinyal clock untuk melakukan operasinya. Modul register menyimpan dan mengeluarkan data, sedangkan modul akumulator mengakumulasi data dan mengeluarkan nilai akumulasi.